Temco Platform

Support intake — engineering · built Nov 12, 2026

Support intake — engineering

How the support window in t3000-support-intake.md is actually built: the stack, the contract between the two processes, and the method for each part. The intake document decides *what* exists; this one decides *how*.

Every technology named here is already in the repository it belongs to. Nothing new is added to any of the three products except source files.

1. The stack at a glance

Layer	Language	Runtime and libraries already present	What it owns
T3000 shell	C++14 (MFC)	CMake, `CMAKE_CXX_STANDARD 14`, WebView2 `1.0.1150.38` (x86), jsoncpp (`Json-cpp/include`), GDI+ (`mygdiplus.h`)	capture, context, logs, outbox, handoff, crash marker
T3000 web UI	TypeScript 5.8, React 18.3	Vite under Quasar 2, React Router 6.22, Fluent UI v9 (`@fluentui/react-components`), Vitest	the Support page: four steps, annotation, Help & Learn
T3000 local service	Rust	axum 0.7, tokio, tokio-tungstenite, rusqlite — HTTP on 9103, WebSocket on 9104	serves the bundle; already the localhost origin the window is loaded from
Temco Platform	TypeScript	VS Code workbench fork, the `src/temco` shell (43 KB), `temco-*` extensions with webview panels	the inbox, the issue draft, the forum draft
Temco Learn	Markdown, DocFX	the site's own generator	the forum topic, after approval
Automation	YAML + shell	GitHub Actions, `GITHUB_TOKEN` only	labels, dedupe, one comment, the docs-owed flag

Three languages, and each is already the language of the thing it runs in. There is no fourth runtime to install, and no new dependency in any manifest.

---

2. Why each language sits where it does

Question	Answer
why is the capture C++?	it is the only process that owns the window handle, the log paths and the device list. A page cannot photograph its own host
why is the window a React page?	the app already ships a React 18 + Fluent UI surface (`t3-react`), with the Documentation page as the working precedent
why is the queue in the platform, in TypeScript?	it is a webview panel in a module that already exists; a support inbox is another panel, not another program
why is there no server?	every path in the intake document finishes at GitHub's own form or its API, and both accept a request from a browser or a workflow
why is the local Rust service not used for capture?	it has no window handle. It stays the origin that serves the page, which is what it already is

---

3. The message contract

3.1 The envelope that already exists

The page and the shell do not trade free text. `HandleWebViewMsg` (`T3000/BacnetWebView.cpp`) parses a JSON object, switches on an **integer** `action` , and echoes the caller's `msgId` back:

```JSON
{ "action": 41, "msgId": "7" }
```

The reply carries the action name plus `_RES` :

```JSON
{ "action": "SUPPORT_CAPTURE_RES", "msgId": "7", "ok": true, "path": "D:\\T3000\\Databas
```

So the support messages are **new enumerators on the existing enum**, appended at the end so no existing number moves:

C++

```
enum WEBVIEW_MESSAGE_TYPE
{
        // ... every existing entry, unchanged ...
        SUPPORT_CONTEXT = 100,
        SUPPORT_CAPTURE,
        SUPPORT_LOGS,
        SUPPORT_SETTINGS,
        SUPPORT_CLIPBOARD,
        SUPPORT_HANDOFF,
        SUPPORT_OPEN_URL,
        SUPPORT_OUTBOX,
        SUPPORT_OUTBOX_RES,
};
```

Rule	Why
append, never renumber	the numbers are the wire format, and an old bundle with a new EXE must not call the wrong handler
every reply is `{ action: "X_RES", msgId, ok }`	the channel is shared with the rest of the app; a reply without an id cannot be matched to its request
`ok: false` carries `error` and never a stack	the page shows one sentence and the reason code
the page feature-detects by *asking*, not by version	a bundle can be newer than the EXE; a missing action answers `ok: false, error: "unknown_action"` and the page hides what needs it

3.2 The eight messages

Action	Request fields	Reply
`SUPPORT_CONTEXT`	—	`version`, `build`, `os`, `locale`, `screen`, `panels[]` (each `name`, `firmware`, `product`)
`SUPPORT_CAPTURE`	`mode`: `whole` | `client` | `screen`	`path`, `width`, `height`, `bytes`
`SUPPORT_LOGS`	`lines` (default 200)	`path`, `bytes`, `dropped` (lines removed at the top)
`SUPPORT_SETTINGS`	—	`path`, `bytes`, or `ok: false, error: "withheld"`
`SUPPORT_CLIPBOARD`	`path`	`ok` — whether the PNG reached the clipboard
`SUPPORT_HANDOFF`	`bundle` (folder), `summary`	`ok`, `route`: `platform` | `browser`
`SUPPORT_OPEN_URL`	`url` (https only)	`ok`
`SUPPORT_OUTBOX`	`op`: `save` | `list` | `send` | `delete`, `bundle`	`items[]` for `list`; `ok` otherwise

3.3 Threading, and the one trap

`HandleWebViewMsg` runs on the UI thread holding `handleWebViewMsgMutex`. A capture takes tens of milliseconds and the log copy takes more. So:

- the **reply is built after the work**, but the work itself is dispatched to a worker thread and the reply posted back with `PostWebMessageAsJson` when it finishes — the page is already written against a promise with a timeout (§5.3), so a slower reply costs nothing;
- the capture **must** run on the thread that owns the window, because `PrintWindow` needs the window's message loop to be answering — so the pump is kept free by not blocking the lock: capture on the UI thread but outside `handleWebViewMsgMutex`;
- the log and settings copies are plain file I/O and run on any thread.

---

4. The five host methods

Each one is a small function in one new translation unit. Nothing else in `T3000/` changes.

4.1 Capture — `SupportCapture.cpp`

The whole method is: get the window's real rectangle, ask it to paint itself into a memory DC with the full-content flag, encode with GDI+, save.

C++

```cpp
bool CaptureWindow(HWND hwnd, const std::wstring& path, int& width, int& height)
{
	RECT frame = {};
	if (FAILED(DwmGetWindowAttribute(hwnd, DWMWA_EXTENDED_FRAME_BOUNDS, &frame, siz
		GetWindowRect(hwnd, &frame);
	}
	width = frame.right - frame.left;
	height = frame.bottom - frame.top;
	if (width <= 0 || height <= 0) {
		return false;
	}

	HDC screen = GetDC(nullptr);
	HDC memory = CreateCompatibleDC(screen);
	HBITMAP dib = CreateCompatibleBitmap(screen, width, height);
	HGDIOBJ previous = SelectObject(memory, dib);

	const BOOL painted = PrintWindow(hwnd, memory, PW_RENDERFULLCONTENT);

	CLSID png;
	CLSIDFromString(L"{557CF406-1A04-11D3-9A73-0000F81EF32E}", &png);
	Gdiplus::Bitmap bitmap(dib, nullptr);
	const Gdiplus::Status saved = bitmap.Save(path.c_str(), &png, nullptr);

	SelectObject(memory, previous);
	DeleteObject(dib);
	DeleteDC(memory);
	ReleaseDC(nullptr, screen);
	return painted && saved == Gdiplus::Ok;
}
```

Detail	Decision
the flag	`PW_RENDERFULLCONTENT` (`0x2`) is what makes a WebView2 child appear at all; without it the pane is black
the rectangle	`DwmGetWindowAttribute(DWMWA_EXTENDED_FRAME_BOUNDS)` excludes the invisible resize border and keeps a multi-monitor window on its own monitor; `GetWindowRect` is the fallback
DPI	the app declares no DPI awareness — no manifest entry and no `SetProcessDpiAwareness` call — so the rectangle it reports is already in the pixels it draws in. The capture therefore never rescales, and the page is told the pixel size it received
the encoder	GDI+ is already linked and used (`Gdiplus::Font` in `BacnetGraphic.cpp`); PNG needs no new library
the cap	if the long edge exceeds 2560 px, scale once with `Gdiplus::Bitmap::GetThumbnailImage` and re-encode
where it saves	`<exe>\Database\Support\<timestamp>\window.png` , beside the app's own data (`g_strExePth`), never `%TEMP%` — plant PCs clean `%TEMP%` and support loses the file
black output	if more than 99 % of pixels are one colour, capture again with `client` mode and report `ok: false, error: "capture_blank"` rather than sending a black image

New link dependency: `dwmapi.lib` for the frame bounds. One line in `CMakeLists.txt` , next to the WebView2 loader.

4.2 Context — `SupportContext.cpp`

Read, do not compute: the version from the app's own version resource, the screen name from the frame's current view, the panels from the already-loaded device list in memory.

Field	Source
`version` , `build`	the executable's `VS_VERSIONINFO` , as About already reads it
`os` , `locale`	`RtlGetVersion` and `GetUserDefaultLocaleName` — not `GetVersionEx` , which lies without a manifest
`screen`	the frame's active view title, which the page can cross-check against its own route
`panels[]`	the same list the webview is already given by `GET_PANELS_LIST` , filtered to those with data on screen

The reply is capped at 4 KB. A panel name is user data, so the field is `name` and nothing else is read from the device.

4.3 Logs — `SupportLog.cpp`

Read the tail, do not load the file: seek to `max(0, size - 512 KB)`, discard the first partial line, keep the last 200 lines, write UTF-8, cap at 256 KB, and record how many lines were discarded in `dropped`.

Detail	Decision
encoding	the app's logs are locale-encoded; the excerpt is converted to UTF-8 once, so the page never guesses
redaction	the excerpt is **not** rewritten — the review panel (§8.7 of the intake) shows it whole and lets the person remove the attachment
failure	a missing log is `ok: true` with `path: null` and `bytes: 0`, not an error: a report without logs is still a report

4.4 Outbox — `SupportOutbox.cpp`

A report is a folder, so it survives a crash between any two writes:

JSON

```
{
  "id": "2026-11-12T09-11-42",
  "created": "2026-11-12T09:11:42Z",
  "title": "Trendlog graph is empty after I picked 6 points",
  "kind": "bug",
  "state": "queued",
  "files": ["window.png", "t3000-2026-11-12.log"],
  "context": { "version": "2026.0912", "screen": "Devices > Trendlogs" }
}
```

Operation	Method
`save`	write `report.json.tmp`, flush, rename to `report.json` — a rename is atomic on NTFS, so a half-written report is never listed
`list`	enumerate `<exe>\Database\Support\*` for folders with a `report.json`; ignore the rest
`send`	the same handoff as `SUPPORT_HANDOFF`, then `state: "sent"`
`delete`	delete the folder, then report the new list

A `state` field rather than a move, so a folder never disappears under a running send.

4.5 Handoff — `SupportHandoff.cpp`

One function, two routes, and the second is not a fallback in the apologetic sense — it is the normal path on a machine without the platform:

Step	What happens
1	`ShellExecuteEx` with `temco-platform://support/new?bundle=<url-encoded folder>`
2	`ShellExecuteEx` returns a value above 32 when the protocol is registered; anything else means no handler
3	on no handler: put the PNG on the clipboard as `CF_DIB`, then open `reportIssueUrl` with `?title=` and `?body=` prefilled
4	reply `route: "platform"` or `route: "browser"`, so the page can say which one happened

Detail	Decision
no shell injection	the URL is built from a fixed scheme, a fixed path and one percent-encoded local path. No user text reaches the command line
the clipboard	`CF_DIB` (not `CF_HDROP`) — a browser accepts a pasted DIB into a file input, and GitHub's attachment box is a file input
the size	a prefilled GitHub URL cannot carry a 400 KB image; the URL carries text only, and the image travels by clipboard

4.6 The crash marker

`SetUnhandledExceptionFilter` writes one small file before the process dies, and the next start reads it.

```JSON
{ "fingerprint": "0xC0000005:trendlog+0x1f42", "first": "2026-11-12T09:11:03Z", "count":
```

Detail	Decision
the fingerprint	exception code plus module and offset — stable across runs, distinct across faults, which is what `Do not ask again` needs
what the handler writes	200 bytes, with `CreateFileW(..., FILE_FLAG_WRITE_THROUGH)`. It does **not** read the log, build JSON with jsoncpp, or allocate: a handler that allocates can fault again
the full dump	left to Windows Error Reporting's `LocalDumps` key, configured by the installer, not by the app
the counter	`count` is why the third crash of the same fault is reported without asking again

5. The Support page

5.1 Where it goes

The app already carries a React surface, so the window is a feature folder, not a new app:

Path	What it is
`src/t3-react/features/support/pages/SupportPage.tsx`	the window: header, rail, step content
`src/t3-react/features/support/components/`	`ScreenshotStep`, `AnnotationCanvas`, `DetailsStep`, `ReviewStep`, `SentStep`, `OutboxStep`, `HelpSearch`
`src/t3-react/features/support/services/supportBridge.ts`	the promise layer over the message contract (§5.3)
`src/t3-react/features/support/support.css` or `.module.css`	Fluent tokens only; no hard-coded colour
`src/t3-react/app/router/routes.ts`	one lazy route, in the same table as `DocumentationPage`

The route is registered exactly like the Documentation page is: `lazy()` import, inside the existing router table. No new build target, no new entry point — `npm run build` in the web project picks it up, and `npm run deploy:t3000` copies the result into `T3000/ResourceFile/webview/www` as it does today.

5.2 Components, not inventions

Need	Use
buttons, fields, dropdowns, badges	`@fluentui/react-components` — already a dependency, and it is the same design system the Documentation page uses
light and dark	the app's `theme` folder and its existing Fluent theme provider
icons	`@fluentui/react-icons`, already a dependency
the step rail	a `<nav>` with `aria-current="step"`; no custom list widget
the annotation surface	one `<canvas>` with pointer events; `fabric` and `paper` are already in the app, but plain canvas is enough and is 200 lines
translation	the app's existing i18n pipeline (XLIFF, `xliff` is already a dependency); no second translation system

5.3 The bridge

One module, one function, and the page never posts a raw message anywhere else:

TYPESCRIPT

```typescript
type Reply = { msgId?: string; ok?: boolean; action?: string; error?: string; [key: stri

const pending = new Map<string, (reply: Reply) => void>();
let sequence = 0;

export function request(action: number, fields: Record<string, unknown> = {}, timeoutMs
        const msgId = String(++sequence);
        return new Promise<Reply>((resolve, reject) => {
                const timer = setTimeout(() => {
                        pending.delete(msgId);
                        reject(new Error('bridge_timeout'));
                }, timeoutMs);
                pending.set(msgId, reply => {
                        clearTimeout(timer);
                        pending.delete(msgId);
                        resolve(reply);
                });
                window.chrome.webview.postMessage(JSON.stringify({ action, msgId, ...fi
        });
}

window.chrome.webview.addEventListener('message', event => {
        const reply = typeof event.data === 'string' ? JSON.parse(event.data) as Reply
        if (reply.msgId) {
                pending.get(reply.msgId)?.(reply);
        }
});
```

Rule	Why
every call has a timeout	a handler that throws must not leave a spinner forever
`request` is the only writer to `chrome.webview`	the channel is shared with the rest of the app, and a second writer means two listeners and duplicate replies
no reply is trusted for shape	the reply is parsed and its fields are checked before use, because it crosses a process boundary
the browser build still loads	`window.chrome?.webview` is undefined in a browser; the module exposes `isHosted()` and the page runs read-only in a browser for development

5.4 The annotation canvas

Step	Method
load	`request(SUPPORT_CAPTURE, { mode })` → a path → `<img src>` served by the local Rust origin, or the host's own `file://`-equivalent route
draw	keep a base image and a list of annotations `{ tool, from, to, width }`; redraw on every pointer move (60 fps on a 1600 × 900 canvas is not a problem)
redact	fill the rectangle with black **on the export canvas**, not only on screen; the exported PNG has no pixels there
export	`canvas.toBlob('image/png')`, then hand the blob back to the host as a path — the host writes it, because the page cannot write to disk
undo	pop the annotation list; the base image is never mutated

The one rule that is not negotiable: **redaction happens before the file is exported**, so no un-redacted copy exists on disk.

5.5 Help & Learn and the offline copy

The window is loaded from `http://localhost:9103`, which is served by the Rust service that is already running. So the help copy is:

Piece	Method
the copy	the site's content package, unpacked beside the app during install — the same tree the site is generated from
search	an index built at bundle time (DocFX already emits one); the page filters it in memory, which is why search works with no network
the online half	`SUPPORT_OPEN_URL` opens the real site in the browser
if the service is down	the page loads from the bundle's own files (`chrome-extension://`-style local origin) and hides search; the report half still works

6. The platform surfaces

Surface	Where	Method
the protocol handler	`src/temco/platform/` (one file, one registration)	`IURLService.registerHandler` with an `IURLHandler` whose `handleURL` claims `support` + `/new` and executes a command
the inbox panel	`extensions/temco-workbench/src/support.ts`	`vscode.window.createWebviewPanel`, in the same shape as `dashboard.ts` — one panel map, one `open()`
the issue draft	the panel writes `draft.md`	a `WorkspaceEdit`, so the draft is a real file a person can read, diff and copy
the forum draft	the panel, after an approval	nothing is posted; the panel shows the text as it would appear

TYPESCRIPT

```
class SupportUrlHandler implements IURLHandler {
        constructor(private readonly commandService: ICommandService) { }

        async handleURL(uri: URI): Promise<boolean> {
                if (uri.authority !== 'support' || uri.path !== '/new') {
                        return false;
                }
                const bundle = new URLSearchParams(uri.query).get('bundle') ?? undefine
                await this.commandService.executeCommand('temcoWorkbench.support.open',
                return true;
        }
}
```

Rule	Why
the handler **returns** `false` for any other URL	a protocol handler that swallows every `temco-platform://` URL breaks the app's own links
the bundle path is treated as untrusted	it arrives from another process. It is resolved, must be inside the app's `Database\Support` folder, and its files are read with a size cap; nothing is executed and nothing is extracted
the panel loads no remote content	a strict CSP, a nonce for the one inline script, and images only through `asWebviewUri`
one command, one panel	reopening the same bundle focuses the existing tab rather than opening a second copy

7. Automation

Piece	Method
the workflow	`.github/workflows/support-triage.yml`, `on: issues: [opened, edited]`, `permissions: issues: write`
the token	`secrets.GITHUB_TOKEN` only. No PAT, no app key, no repository secret to rotate
classify	a table in the workflow: label keywords → `area:trendlogs`, `area:bacnet`, `area:graphics`; the issue template's own labels do the type
dedupe	`gh issue list --search "<two distinctive words>" --state all`, then link the closest rather than guess
the comment	one comment, posted once, with an `<!-- support-triage -->` marker so a re-run edits instead of stacking
the draft	a Copilot/LLM step only when a repository variable enables it; the fallback is the same comment without prose
docs-owed	when the body names a help page id, add `docs-owed`; that label already exists for the site's own checks
the forum	never automated. The draft is approved by a person (§9.3 of the intake)

8. Nothing new is installed

Layer	New dependency
C++	`dwmapi.lib` — a Windows SDK import library, not a package
web UI	none: React, Fluent UI, Router, Vitest and canvas are all present
Rust	none: the service is unchanged, and no support route is added to it in phase 1 or 2
platform	none: `IURLService`, webview panels and the command registry are the fork's own
automation	none: `gh` is preinstalled on GitHub's runners

If a later phase wants one-click anonymous filing (§13 of the intake), the smallest addition is a single Cloudflare Worker or Azure Function — not a service in this repository, and not a token in the client.

---

9. Failure modes and what the user sees

Failure	Detection	What the page does
the web bundle is newer than the EXE	`SUPPORT_CONTEXT` answers `unknown_action`	hides the capture step, keeps the form, hands off through the browser route
the capture is black	the host's pixel check	says so, offers `client` mode, and offers the whole-screen capture
the log is missing	`path: null`	the attachment is absent from the review, with one line saying why
no platform installed	`ShellExecuteEx` ≤ 32	the browser route, with the clipboard message
no connection	the page's own `navigator.onLine` and the first failed handoff	saves to the outbox and says when it will leave
the outbox is full	more than 20 queued reports	refuses a new save and offers to send or delete
the local service is down	the page's fetch fails	renders from bundled files, search hidden, report intact
the platform is given a bad bundle	the path escapes the support folder, or a file is over the cap	the panel shows the problem and offers the log, and posts nothing

Every one of these is a sentence in the window, not a dialog with an error code.

---

10. Budget

Thing	Target	Why
the page's own assets	≤ 40 KB gzipped	it joins a 33 MB bundle; this is noise, and a feature folder that pulls in a chart library would not be
cold open	≤ 200 ms on a 1366 × 768 plant PC	it opens from localhost, with no network
the capture	≤ 120 ms for a 1600 × 900 window	it runs on the UI thread
the PNG	≤ 1.5 MB, long edge ≤ 2560 px	it is pasted into a browser or attached to an issue
the bundle	≤ 2 MB zipped	the outbox is a folder on a plant PC's disk
the platform panel	≤ 150 ms to first paint	it reads one folder of local files

11. How each part ships

Artifact	Built by	Reaches the customer by
`T3000.exe` with the support module	CMake, the existing target — one added source file and `dwmapi.lib`	the existing installer
the web bundle with the Support page	`npm run build` in the web project, then `npm run deploy:t3000`	the same bundle path the Documentation page already uses, and T3000's own updater
the offline help copy	the site's content package	the installer
the platform panel	the extension build	the platform's normal release
the workflow	committed YAML	takes effect on merge

The only version handshake is the one the page performs itself: it asks the host what it can do and adapts. There is no separate protocol version to keep in step.

12. Work breakdown

In the order that keeps every step shippable.

#	Task	Where	Done when
1	`SUPPORT_CONTEXT` and `SUPPORT_HANDOFF`, browser route only	`T3000/Support*.cpp`, `BacnetWebView.cpp`	Help ▸ Report an Issue opens GitHub with the title prefilled
2	`SUPPORT_CAPTURE` and `SUPPORT_CLIPBOARD`	`T3000/SupportCapture.cpp`, `dwmapi.lib`	the screenshot pastes into the issue body from the clipboard
3	the Support page, steps 1–4, no annotation	`src/t3-react/features/support/`	a report is written in the window and handed off
4	the annotation canvas and redaction	the same folder	a redacted area is gone from the exported PNG
5	`SUPPORT_LOGS`, `SUPPORT_SETTINGS` and the review panel	C++ and the page	an attachment can be read and removed before sending
6	the outbox	`SupportOutbox.cpp`	a report written offline leaves on the next start
7	the crash marker and the next-start prompt	`SupportCrash.cpp`	a killed process produces one prompt, fingerprinted
8	Help & Learn over the offline copy	the page, and the installer's copy	search answers with the network off
9	the protocol handler and the inbox panel	`src/temco`, `extensions/temco-workbench`	a bundle opens as a tab, and the reply is copied by hand
10	the triage workflow	`.github/workflows/support-triage.yml`	a new issue is labelled and answered once

13. Validation

Layer	What proves it
C++	a `support-selftest` switch that captures the main window and writes a PNG, run by hand on a Windows build; the log reader and the outbox have plain unit tests
the page	Vitest, already configured: the bridge's promise, timeout and unknown-action behaviour; the redaction export
the platform	the existing test harness with a fake `IURLService`, and a bundle-path rejection test
the docs	`npm run check-temco-docs`, which already runs in `npm run check-temco`
the end to end	a scripted Windows run: open T3000, trigger a report, confirm the issue opens with title and body, and the clipboard holds the PNG